

Channel select

Channel select module has logic of arbiter to select the channels. This module provides the trigger to DMA engine to initiate the transaction. Channel select module is a modport in the top-level interface. So, all the ports are members of the modport. The channel select module has instances of 8 fifos, priority mux, arbiter and also encoder. The eight fifos are instantiated using generate statements. These fifos are used for sorting priority requests and different channels.

All the fifos are 32 deep and 8 bit wide. The write and read signals will increment or decrement respectively a counter. Depending on the counter the empty or full are decided. Write to the FIFO happens from priority encoder and read happens from arbiter module. The priority encoder has a generate statement in which valid signals from all the channels are asserted depending on the channel configuration and requests. Multiplexer also has generate statements where the selected channels configurations are connected to DMA.

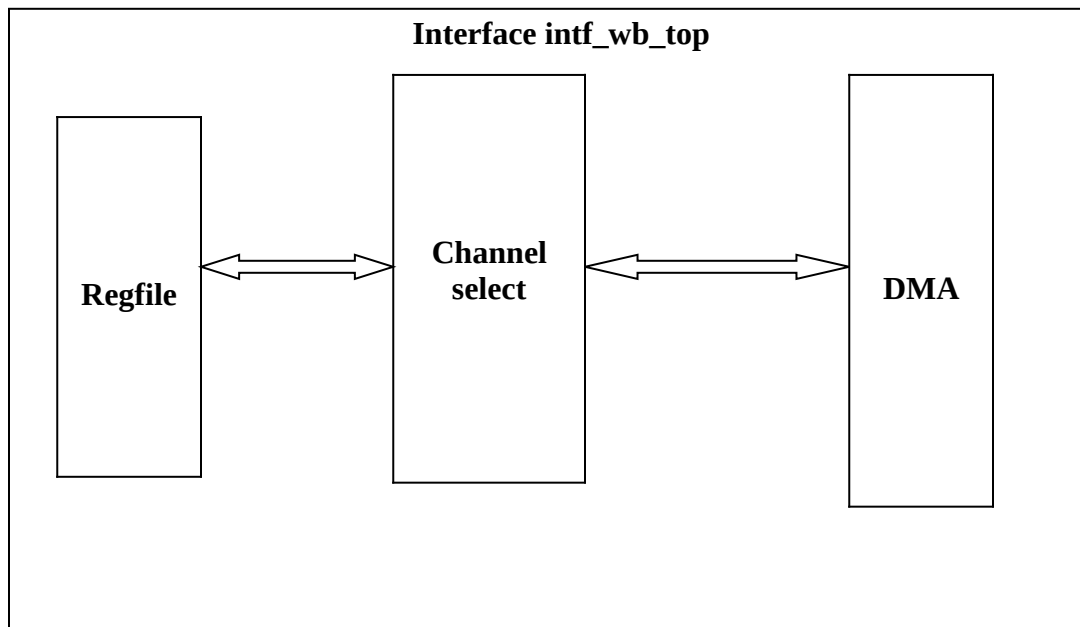


Fig 1. Inter-connect diagram

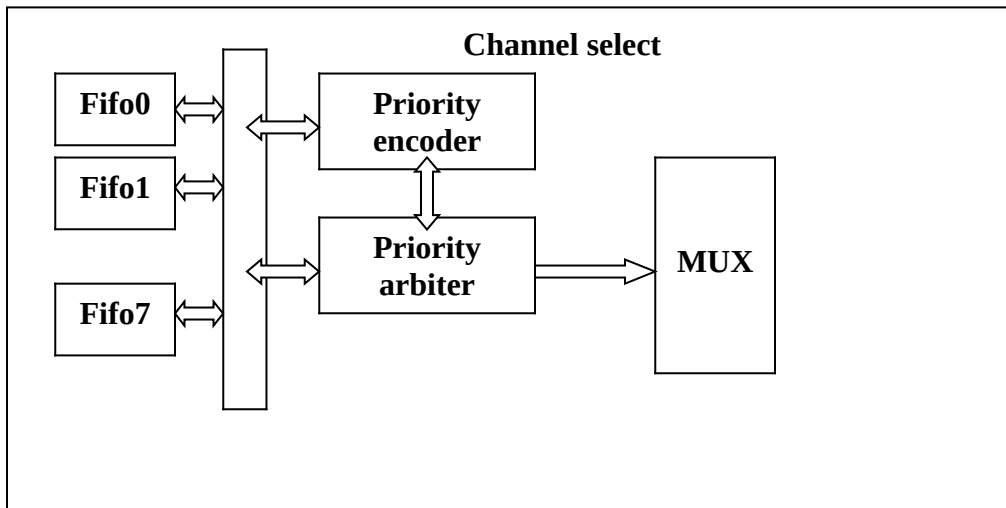


Fig 2. Inside channel select

Priority encoder:

This module sorts the requested channels priorities into different fifos. Once this writing of channels into different fifos is over, the encoder triggers arbiter to start arbitration. A state machine is coded to sort all the requests into fifos.

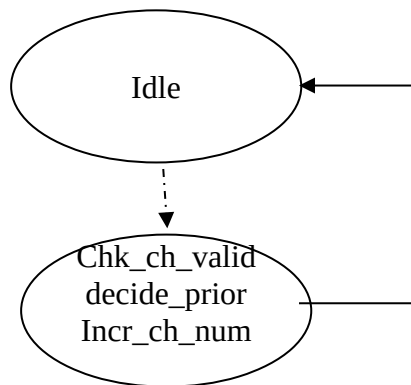


Fig 3 : State machine for priority encoder

State description

Idle:

On reset the state is reached. When there is request from host, the state transition begins.

Idle -> chk_ch_valid : if request from software and assert ch_num = 0

Chk_ch_valid:

Checking of channel validity is performed here.

chk_ch_valid -> decide_prior : if ch_valid[ch_num] == 1

chk_ch_valid -> incr_ch_num : if ch_valid[ch_num] == 0

Depending on the priority value asserted for that channel the channel number is written into that particular fifo.

The channel number is incremented. Also the check if all the channel numbers are verified or not is also performed. While going back to idle the arbiter begin should be asserted.

Channel arbiter:

After getting trigger form encoder, Channel arbiter reads the eight fifos to see and grants the access to channels depending on priority. Once the grant is asserted also trigger the multiplexer to output the channel configuration to dma. This trigger is also connected as trigger to dma. After the next channel signal getting asserted from DMA, the arbiter resumes the above operation again to assert grant for next channel.

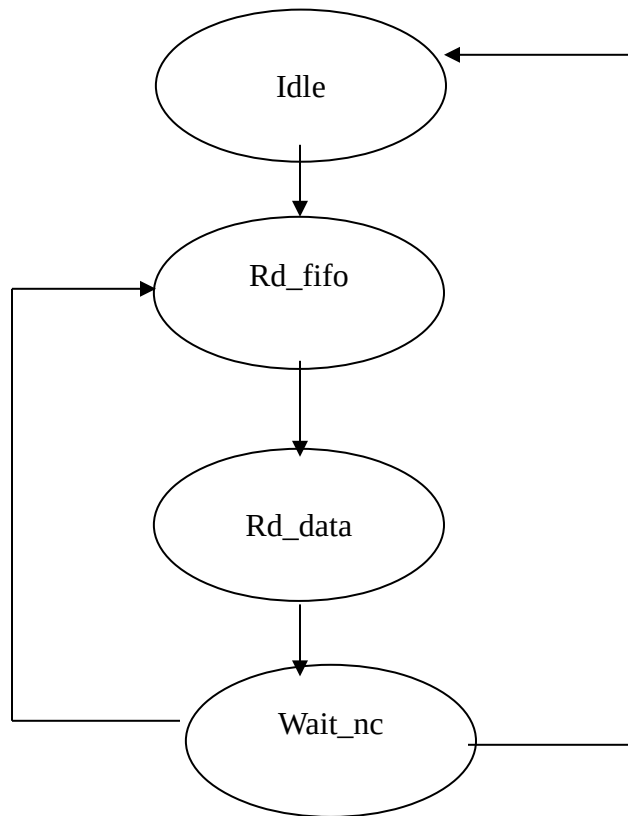


Fig 4 State machine of priority arbiter

Idle :

Until there is trigger from encoder the arbiter will be in this state.

Idle->rd_fifo : on trigger

Rd_fifo:

Assert a read and proceed.

Rd_fifo-> **rd_data** : after asserting read.

Rd_data:

Latches the data here and proceeds.

Rd_data->**wait_nc**: next clock.

Wait_nc:

Waits until the dma asserts next channel.

Wait_nc -> **rd_fifo**: If all the fifos are covered.

Wait_nc-> **idle**: Once all the fifos are covered.

Multiplexer:

Once the arbiter asserts the select lines, the multiplexer outputs the corresponding 8 values of that particular channel. Generate statements are used to output the data of each type to dma.

Signal name	Width	Description
<i>Inputs</i>		
i_clk	1	Global Clock
i_reset	1	Global Reset
i_next_ch	1	From DMA engine, indicating that DMA of current channel is done and next channel can be selected
i_dma_ack	1	Acknowledge from DMA indicating that either chunk transfer or total transfer is done
i_req	32	Input from Host indicating to start the DMA hardware handshake mode
i_nd	1	Input from host indicating DMA engine to fetch the channel descriptors from the memory attached to interface 0
i_dma_busy	1	Indication from DMA engine that, the DMA is busy
i_ch_csr	30*32	Channel CSR value from regfile
i_ch_sz	30*32	Channel transfer size from regfile
i_ch_addr0	30*32	Channel address0 from regfile
i_ch_am0	30*32	Channel address0 mask from regfile
i_ch_addr1	30*32	Channel address1 from regfile
i_ch_am1	30*32	Channel address1 mask from regfile
i_ch_desc	30*32	Channel linked list descriptor pointer from regfile
i_ch_swptr	30*32	Channel software pointer from regfile
<i>Outputs</i>		
o_pointer	32	Linked list descriptor pointer
o_prev_pointer	32	Linked list previous pointer
o_csr	32	Configuration and status register value from regfile

Signal name	Width	Description
o_tx_size	32	Channel transfer size
o_src_addr	32	Source address
o_dest_addr	32	Destination address
o_src_addr_mask	32	Source address mask
o_dest_addr_mask	32	Destination address mask
o_ch_sel	5	selected Channel value
o_ndnr	32	Next descriptor with no request
o_dma_start	1	To DMA engine requesting the start of DMA

Table 1 showing the input output signals of channel select

Signal Name	Width	Description
<i>Inputs</i>		
Chsel modport		
i_prififo_prienc_full	8	
<i>Outputs</i>		
O_start_arbiter	1	Output to arbiter to begin the arbitration
prienc_fifo_data	6*8	
Prienc_fifo_wr	8	

Table 2 showing the input output signals of priority encoder

Signal name	Width	Description
<i>Inputs</i>		
Chsel modport		Modport signals
Prienc_priarb_start	1	Start from enc
Prififo_priarb_empty	8	Empty from fifo
Prifio_priarb_data	6*8	From fifo
<i>Outputs</i>		
Priarb_primux_sel	6	To mux
Priarb_prififo_rd	8	Read to fifo

Table 3 showing the input output interface of channel arbiter